

CrowdFlow 코드 흐름 검토 기록

캘리브레이션 보드 도착 전 검토 항목과 실측 후 수정 항목 분리

1. 검토 기준

- 작성일: 2026-06-18 목요일, Asia/Seoul 기준.
- 다음 주 월요일 작업 시작 예상일: 2026-06-22.
- 검토 범위: main.py -> detector.py -> grid_calculator.py -> dashboard.py 흐름. 보조로 config/app_config.json, settings.py, result_publisher.py, data_logger.py를 확인함.
- 이번 단계 원칙: 코드 편집 없이 흐름을 점검하고, 캘리브레이션 보드 없이 판단 가능한 항목과 실측 후에만 확정할 항목을 분리함.

2. 현재 실행 흐름 요약

단계	파일	현재 동작	판단
1	main.py	AppSettings로 source, model_path, model_type, imgsz, confidence, grid 설정을 읽음. 현재 source는 data/test_video2.mp4, model은 weights/yolo11l_crowdflow.pt, calibration_path는 null.	테스트 영상 기반 실행은 가능하다. 다만 calibration_path가 null이면 실제 밀집도 검증값이 아님.
2	main.py / DroneCamera	DroneCamera에서 첫 프레임을 얻고, 첫 프레임 크기를 기준으로 homography를 설정함.	드론 없이도 테스트 영상으로 파이프라인 확인 가능. 실제 Tello 프레임 안정성 평가는 드론 비행 후 별도 수행.
3	main.py	calibration_path가 있으면 JSON의 src_points/dst_points를 사용하고, 없으면 전체 프레임을 10m x 10m 영역으로 임시 매핑함.	현재 fallback은 개발용이다. 화면에 결과가 보여도 실제 거리/면적 기반 밀집도라고 주장하면 안 된다.
4	detector.py	YOLO 또는 RT-DETR 모델을 로드하고, person/pedestrian/people 이름을 가진 클래스를 자동 탐지 대상으로 사용함. 지정값이 없고 이름 매칭 실패 시 class 0을 사용함.	VisDrone 학습 모델의 실제 class id/name을 확인해야 한다. 보행자와 사람을 둘 다 학습했다면 config에서 명시하는 편이 안전하다.
5	grid_calculator.py	bbox 하단 중심점을 발 위치로 보고 픽셀 좌표를 실제 좌표로 변환한 뒤 2m 격자에 누적함. 밀집도는 count / cell_area로 계산함.	아이디어는 맞다. 정확도는 homography와 실제 영역 크기 입력이 결정한다.
6	result_publisher.py / data_logger.py	분석 결과를 latest_result.json과 SQLite DB에 저장함.	dashboard.py와의 연결 구조가 분리되어 있어 좋다. JSON 원자 저장도 현재 구조에 적합하다.
7	dashboard.py	1초마다 latest_result.json을 읽고 히트맵, 경고, DB 통계를 표시함. calibration_mode가 fallback이면 경고를 띄움.	현재 단계에서 중요한 안전장치가 들어가 있다. 실측 전 결과 오해를 줄이는 역할을 한다.

3. getPerspectiveTransform 사용 판단

- 현재 GridCalculator는 cv2.getPerspectiveTransform(src_pts, dst_pts)를 사용한다.
- 이 방식은 정확히 4개의 대응점으로 평면 투시 변환을 계산할 때 적합하다. 600mm x 400mm 캘리브레이션 보드의 네 모서리를 기준으로 실험한다면 학교에서 배운 방식과도 맞다.
- cv2.findHomography는 4개보다 많은 대응점을 쓰거나, 점 검출 오차와 이상치를 RANSAC으로 걸러야 할 때 더 유리하다.

- 따라서 다음 주 첫 실측은 getPerspectiveTransform 유지가 적절하다. 단, 체커보드 내부 코너를 여러 개 검출해 더 많은 점을 쓰는 방식으로 확장하면 findHomography 검토 가치가 생긴다.

4. 캘리브레이션 보드 도착 전 - 검토만 할 항목

우선순위	항목	확인 방법	완료 기준
상	모델 class 이름 확인	YOLO11/RT-DETR 모델의 model.names를 출력해 class id와 이름을 기록한다. 코드 수정 없이 실행 결과만 확인한다.	person/pedestrian/people 또는 학습한 두 클래스의 id가 명확히 기록되어야 한다.
상	테스트 영상 파이프라인 확인	현재 source인 data/test_video2.mp4와 test_video1.mp4를 각각 실행해 detection -> grid -> JSON -> dashboard 표시가 이어지는지 본다.	latest_result.json 생성, DB 기록, dashboard 표시, fallback 경고 표시가 모두 확인되어야 한다.
상	fallback 결과 해석 제한	calibration_path가 null인 상태의 밀집도 수치는 실제 m2 기준이 아니라는 점을 발표/기록에 명시한다.	문서와 발표에서 현재 밀집도는 '파이프라인 검증용'으로만 표현한다.
중	YOLO11 vs RT-DETR 비교	같은 테스트 영상에서 추론 시간, 탐지 수, 누락/오탐 경향을 비교한다. 새 학습이나 코드 변경 없이 실행 로그만 모은다.	실측 전 후보 모델 1순위와 예비 모델 1개를 정한다.
중	임계값 기록	detection_confidence 0.1, low_confidence_threshold 0.4, level 기준 2/4/6/8 인/m2를 현재 가정값으로 기록한다.	다음 주 실측 후 바꿀 수 있는 '가정값'으로 분리되어 있어야 한다.
중	캘리브레이션 좌표 양식 준비	보드 네 모서리의 픽셀 좌표, 실제 좌표, 촬영 높이, 카메라 각도, 프레임 해상도를 적을 표를 준비한다.	실험 현장에서 빠뜨리지 않고 기록할 항목이 정리되어야 한다.
하	dashboard 사용성 점검	main.py를 먼저 켜고 dashboard.py를 켤 때 데이터 대기, stale warning, fallback warning이 이해되는지 확인한다.	팀원이 보아도 현재 상태를 오해하지 않아야 한다.

5. 2026-06-22 이후 - 실제 측정 후 수정할 항목

우선순위	수정/확정 항목	근거	수정 방향
상	calibration_path 설정	실제 보드 좌표와 화면 픽셀 좌표가 있어야 진짜 m 단위 변환이 가능하다.	config/calibration.example.json을 실제 측정값 파일로 복사하고, app_config.json의 calibration_path를 해당 파일로 지정한다.
상	src_points 순서 검증	점 순서가 섞이면 homography가 뒤틀려 ignored_count가 증가하거나 격자가 뒤집힌다.	현재 관례는 좌상, 우상, 좌하, 우하 순서로 맞추는 것이 안전하다. 파일 내부 주석 또는 측정 기록에도 같은 순서를 적는다.
상	area_width/area_height 확정	밀집도는 count / 실제 면적이다. 실제 실험 영역 크기가 10m x 10m가 아니면 전체 수치가 틀어진다.	실험 장소에서 측정한 가로/세로를 app_config.json에 반영한다.
상	grid_size 확정	현재 2m 격자는 한 칸 4m2다. 실험 공간이 작거나 사람이 적으면 레벨 변화가 둔해질 수 있다.	시연 목적과 장소 크기에 맞춰 1m, 2m 중 결정한다. 최종 레벨 기준과 함께 기록한다.
상	실측 count 검증	탐지 수와 실제 인원 수 차이를 봐야 모델/임계값을 판단할 수 있다.	0명, 1명, 2명, 4명, 혼잡 배치 등 최소 5개 시나리오로 MAE/RMSE를 기록한다.
중	person_classes 명시 여부	VisDrone에서 사람/보행자 두 클래스로 학습했다면 자동 매칭이 실패하거나 한 클래스만 잡을 가능성이 있다.	model.names 확인 후 실제 사람 관련 class id를 app_config.json의 person_classes에 명시한다. 예: [0, 1] 형태는 확인 후에만 사용한다.
중	confidence 조정	detection_confidence 0.1은 작은 사람 탐지에는 유리하지만 오탐이 늘 수 있다.	실측 누락/오탐을 보고 0.1, 0.2, 0.3을 비교해 최종값을 정한다.
중	low_confidence 경고 해석	현재 낮은 평균 confidence 셀은 level을 최소 3으로 올릴 수 있다.	이 경고가 실제 위험인지, 단순 모델 불확실성인지 실험 데이터로 분리한다.
중	Tello 실제 프레임 검증	테스트 영상과 Tello 실시간 영상은 해상도, 흔들림, 압축 노이즈가 다르다.	이륙 후 송출 안정성, 프레임 크기, 지연, 배터리 상태를 로그로 기록한다.
하	dashboard 표현 조정	실측값이 들어오면 dashboard가 최종 발표 화면 역할을 한다.	fallback 경고가 사라지는지, 실제 모델명/추론시간/제외 탐지가 읽기 쉬운지 확인 후 다듬는다.

6. 현재 코드에 대한 엄격한 판단

영역	현재 판단	엄격한 평가
전체 흐름	입력 -> 탐지 -> 좌표 변환 -> 격자 밀집도 -> JSON/DB -> dashboard 흐름은 연결되어 있다.	구조는 적절하다. 다만 실측 homography 전까지는 '밀집도 시스템 완성'이 아니라 '파이프라인 완성'으로 보는 것이 맞다.
탐지 모델	YOLO11과 RT-DETR을 사용할 수 있는 구조이며, 사람 관련 클래스 자동 선택 로직이 있다.	자동 선택만 믿으면 위험하다. VisDrone 학습 class id/name 확인이 반드시 필요하다.
좌표 변환	getPerspectiveTransform으로 4점 평면 변환을 계산한다.	첫 실험에는 적절하다. 정확도는 코드보다 측정 품질에 더 크게 좌우된다.
밀집도 산정	bbox 하단 중심을 발 위치로 사용해 격자에 누적한다.	균중 밀집도 시스템 방향과 맞다. 다만 드론 각도, 사람 가림, 그림자, 프레임 흔들림은 실측 검증 전까지 미해결이다.
대시보드	fallback 경고, stale warning, 통계 표시가 있다.	시연 준비에는 충분하다. 실제 실험 후 UI 문구와 경고 기준을 데이터에 맞춰 다듬어야 한다.

7. 다음 작업 순서 제안

- 2026-06-18 ~ 2026-06-21: 코드 편집 없이 모델 class 이름, 테스트 영상 파이프라인, dashboard 표시, fallback 경고를 확인한다.
- 2026-06-22: 캘리브레이션 보드 수령 후 네 모서리 기준으로 첫 calibration JSON을 만든다.
- 2026-06-22 이후: 실제 영역 크기, grid_size, person_classes, confidence threshold를 실측 데이터로 확정한다.
- 드론은 이륙 상태에서 송출 안정성이 유지된다는 전제를 두고, 안전 허가와 장소 확보 후 Tello 실시간 테스트를 따로 수행한다.

결론: 현재 방향은 잘못 잡은 것이 아니다. 핵심 미완성 지점은 모델 코드가 아니라 실제 평면 좌표계 확정이다. 캘리브레이션 보드 도착 전에는 새 기능을 추가하기보다, 현재 파이프라인이 결과를 일관되게 저장하고 표시하는지 확인하는 것이 가장 효율적이다.